logo.png

RAPPORT TECHNIQUE COMPLET

SYSTÈME MULTI-RTSP DYNAMIQUE

Serveur C++ (GStreamer) + Interface Python (PyQt) + Rechargement dynamique

Réalisé par :	Ayoub Smayen
Fonction :	Développeur C++
Spécialisation :	Streaming vidéo, architectures embarquées
Version :	2.0
Date :	Avril 2026

Document confidentiel - Tous droits réservés

Table des matières

1 Introduction **5**
1.1 Contexte et objectifs 5
1.2 Portée du document 5

2 Étude d'existant **6**
2.1 Solutions existantes 6
2.2 Conclusion de l'étude 6

3 Besoins Fonctionnels **7**
3.1 Gestion des canaux 7
3.2 Configuration 7
3.3 Interface graphique 7
3.3.1 Administration (Tkinter) 7
3.3.2 Visualisation (PyQt + OpenCV) 8

4 Besoins Non Fonctionnels **9**

5 Architecture du Système **10**
5.1 Vue d'ensemble 10
5.2 Stack technologique 11
5.3 Pipeline GStreamer par canal 11

6 Diagramme de Cas d'Utilisation **12**
6.1 Acteurs 12
6.2 Cas d'utilisation 12
6.2.1 Administrateur 12
6.2.2 Spectateur 12

7 Diagramme de Séquence **13**
7.1 Rechargement automatique - Serveur 13
7.2 Rechargement automatique - Visualisation 14

8 Diagramme de Classes **15**
8.1 Classes C++ (Serveur) 15
8.2 Classes Python (GUI Admin) 15
8.3 Classes Python (Visualisation PyQt) 15

9 Format du Fichier de Configuration **16**
9.1 Configuration du serveur 16
9.2 Paramètres des canaux 16
9.3 Configuration de la visualisation 16

10 Installation et Déploiement **18**
10.1 Prérequis 18
10.1.1 Windows 18
10.1.2 Linux (Ubuntu/Debian) 18
10.1.3 macOS 18
10.2 Compilation du serveur C++ 18
10.2.1 Windows 18
10.2.2 Linux / macOS 18
10.3 Lancement 19

11 Plan de Tests **20**

12 Conclusion **21**
12.1 Bilan du projet 21
12.2 Points forts 21
12.3 Apport personnel 21
12.4 Évolutions prévues 21

Annexes **23**
Annexe A : Extrait de code – Serveur C++ 23
Annexe B : Extrait de code – Visualisation PyQt 23
Annexe C : CMakeLists.txt 24

Table des figures

5.1 Architecture générale du système 10

7.1 Diagramme de séquence - Rechargement serveur 13
7.2 Diagramme de séquence - Rechargement visualisation 14

Liste des tableaux

2.1 Comparaison des solutions existantes 6

3.1 Besoins fonctionnels - Gestion des canaux 7
3.2 Besoins fonctionnels - Configuration 7
3.3 Besoins fonctionnels - Interface admin 7
3.4 Besoins fonctionnels - Interface visualisation 8

4.1 Besoins non fonctionnels 9

5.1 Stack technologique 11

6.1 Acteurs du système 12
6.2 Cas d'utilisation - Administrateur 12
6.3 Cas d'utilisation - Spectateur 12

8.1 Classes C++ du serveur 15
8.2 Classes Python - Administration 15
8.3 Classes Python - Visualisation 15

9.1 Paramètres des canaux 16

11.1 Plan de tests 20

1 Introduction

1.1 Contexte et objectifs

Le protocole RTSP (Real Time Streaming Protocol) est un standard de l'industrie pour la diffusion de médias en temps réel. Ce projet consiste à développer une solution complète de diffusion et visualisation multi-flux RTSP.

Le système se décompose en deux parties indissociables :

- **Un serveur RTSP multi-canal** développé en **C++17 avec GStreamer**
  - Diffusion de flux vidéo (tests, fichiers, caméras)
  - Rechargement à chaud de la configuration
  - Incrustation texte et horodatage par canal
- **Une interface de visualisation dynamique** développée en **Python / PyQt + OpenCV**
  - Chargement automatique des caméras depuis un fichier JSON
  - Mise à jour des flux sans redémarrage
  - Thread par caméra pour garantir la fluidité

1.2 Portée du document

Ce document couvre :

- L'étude d'existant des solutions similaires
- Les besoins fonctionnels et non fonctionnels
- Les diagrammes UML (cas d'utilisation, séquence, classes)
- L'architecture technique détaillée
- Les spécifications d'implémentation
- Les instructions d'installation et de déploiement
- Le plan de tests

2 Étude d'existant

2.1 Solutions existantes

Le tableau 2.1 présente une comparaison des solutions existantes sur le marché.

Table 2.1 – Comparaison des solutions existantes

Solution	**RTSP**	**Multi-flux**	**UI**	**Reconfigurable**	**Customisable**
VLC Media Player					
OpenCV seul					
ZoneMinder					
FFmpeg + SDL					
OBS Studio					
Notre solution					

2.2 Conclusion de l'étude

Aucune solution open-source simple ne combine :

- Multi-RTSP temps réel
- Interface utilisateur moderne
- Rechargement dynamique sans coupure
- Extensibilité (C++ pour le cœur, Python pour l'UI)

D'où la pertinence de notre architecture hybride.

3 Besoins Fonctionnels

3.1 Gestion des canaux

Table 3.1 – Besoins fonctionnels - Gestion des canaux

ID	Fonctionnalité	Description	Priorité
BF-01	Création de canal	Ajouter un nouveau canal RTSP	Haute
BF-02	Suppression de canal	Retirer un canal actif	Haute
BF-03	Édition de canal	Modifier les paramètres	Haute
BF-04	Activation/désactivation	Activer/désactiver sans supprimer	Moyenne
BF-05	Incrustation texte	Texte personnalisé sur chaque flux	Haute
BF-06	Horodatage vidéo	Heure en temps réel sur le flux	Moyenne

3.2 Configuration

Table 3.2 – Besoins fonctionnels - Configuration

ID	Fonctionnalité	Description	Priorité
BF-07	Chargement config	Lecture JSON au démarrage	Haute
BF-08	Rechargement à chaud	Détection modifications	Haute
BF-09	Persistance	Sauvegarde dans le fichier	Haute
BF-10	Port personnalisable	Configuration du port RTSP	Haute

3.3 Interface graphique

3.3.1 Administration (Tkinter)

Table 3.3 – Besoins fonctionnels - Interface admin

ID	Fonctionnalité	Description	Priorité
BF-11	Tableau de bord	État des canaux en temps réel	Haute
BF-12	Ouverture VLC	Lancer VLC avec l'URL du canal	Haute
BF-13	Copie URL	Copier l'URL dans le presse-papier	Moyenne
BF-14	Journal système	Événements serveur en temps réel	Moyenne
BF-15	Démarrer/Arrêter	Contrôler le serveur C++	Haute

3.3.2 Visualisation (PyQt + OpenCV)

Table 3.4 – Besoins fonctionnels - Interface visualisation

ID	Fonctionnalité	Description	Priorité
BF-16	Affichage grille	Visualiser plusieurs flux simultanément	Haute
BF-17	Auto-reload config	Détection changements JSON	Haute
BF-18	Thread par flux	Isolation des flux	Haute
BF-19	Reconnexion auto	Reconnexion si flux perdu	Moyenne

4 Besoins Non Fonctionnels

Table 4.1 – Besoins non fonctionnels

ID	Catégorie	Description	Mesure
BNF-01	Performance	Latence RTSP	$<$ 200ms
BNF-02	Scalabilité	Canaux simultanés	10 streams 720p
BNF-03	Disponibilité	Uptime du serveur	99,5%
BNF-04	Rechargement	Délai prise en compte config	$<$ 3 secondes
BNF-05	Compatibilité	Lecteurs RTSP standards	VLC, FFplay, OBS
BNF-06	Portabilité	Multiplateforme	Windows, Linux, macOS
BNF-07	Sécurité	Authentification	Optionnelle
BNF-08	Maintenabilité	Code modulaire	C++17 + Python 3.10
BNF-09	Utilisabilité	Interface intuitive	$<$ 5 min prise en main
BNF-10	Extensibilité	Sources vidéo futures	Fichier, caméra, réseau

5 Architecture du Système

5.1 Vue d'ensemble

Le système est composé de trois couches principales :

- **Couche Serveur (C++17)** : moteur RTSP, gestion des canaux, watcher de config
- **Couche Configuration (JSON)** : fichier partagé entre serveur et GUI
- **Couche Interface (Python)** : dashboard Tkinter + visualisation PyQt

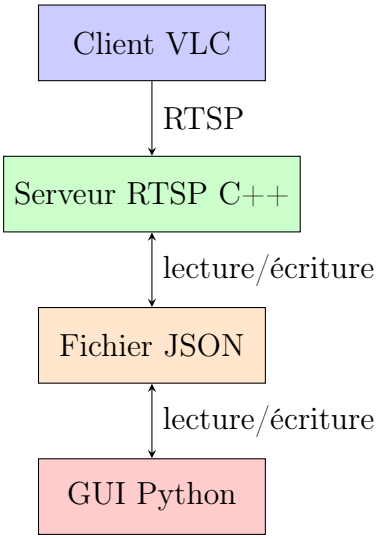

FIGURE 5.1 – Architecture générale du système

5.2 Stack technologique

Table 5.1 – Stack technologique

Composant	Technologie	Rôle
Serveur RTSP	GStreamer + gst-rtsp-server	Traitement vidéo & diffusion
Langage serveur	C++17	Performance & contrôle système
Build système	CMake 3.16+	Compilation multiplateforme
Configuration	JSON	Format lisible, modifiable à chaud
Interface admin	Python + Tkinter	Dashboard de gestion
Interface viewer	Python + PyQt + OpenCV	Visualisation multi-flux
File watcher	watchdog + inotify	Détection modification config
Codage vidéo	x264enc + H.264	Compression
Transport	RTP/UDP	Faible latence

5.3 Pipeline GStreamer par canal

Chaque canal utilise le pipeline GStreamer suivant :

Listing 5.1 – Pipeline GStreamer

```
videotestsrc     video/x-raw     clockoverlay     textoverlay
   videoconvert     x264enc     rtph264pay
```

- **videotestsrc** : source de test générant des motifs vidéo
- **clockoverlay** : incrustation horloge temps réel
- **textoverlay** : incrustation texte personnalisé par canal
- **videoconvert** : conversion espace colorimétrique
- **x264enc** : encodage H.264 faible latence
- **rtph264pay** : encapsulation RTP pour transport RTSP

6 Diagramme de Cas d'Utilisation

6.1 Acteurs

Table 6.1 – Acteurs du système

Acteur	Description
Administrateur	Gère le serveur via la GUI Python
Spectateur	Consomme les flux RTSP via VLC
Visualisateur	Regarde les flux via l'interface PyQt
Système Fichier	Détecte les modifications de config

6.2 Cas d'utilisation

6.2.1 Administrateur

Table 6.2 – Cas d'utilisation - Administrateur

UC-01	Démarrer le serveur RTSP
Pré-condition	Exécutable compilé, config valide
Action	Cliquer 'Démarrer serveur' dans la GUI
Post-condition	Serveur actif, canaux disponibles
UC-02	Ajouter un canal
Pré-condition	Serveur démarré
Action	Remplir le formulaire canal
Post-condition	Canal disponible à l'URL RTSP
UC-03	Modifier la configuration en direct
Action	Modifier rtsp_config.json
Post-condition	Changements appliqués en < 3s

6.2.2 Spectateur

Table 6.3 – Cas d'utilisation - Spectateur

UC-06	Visionner un flux RTSP
Pré-condition	Serveur en cours, URL connue
Action	Ouvrir VLC > Flux réseau > saisir URL
Post-condition	Flux vidéo affiché avec overlay

7 Diagramme de Séquence

7.1 Rechargement automatique - Serveur

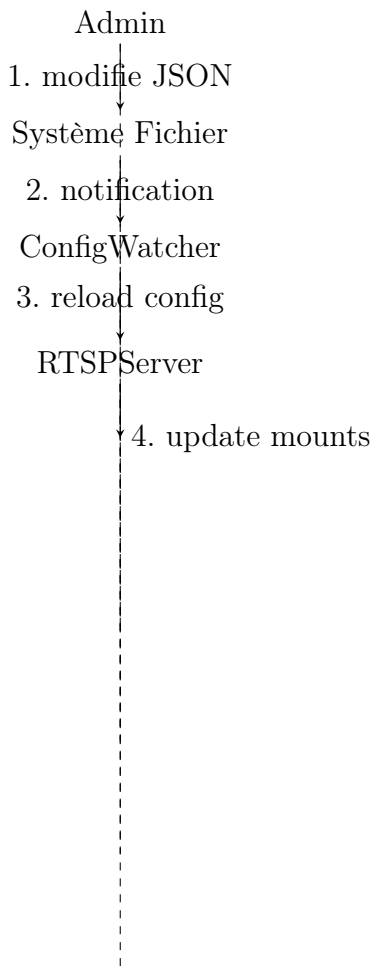

FIGURE 7.1 – Diagramme de séquence - Rechargement serveur

7.2 Rechargement automatique - Visualisation

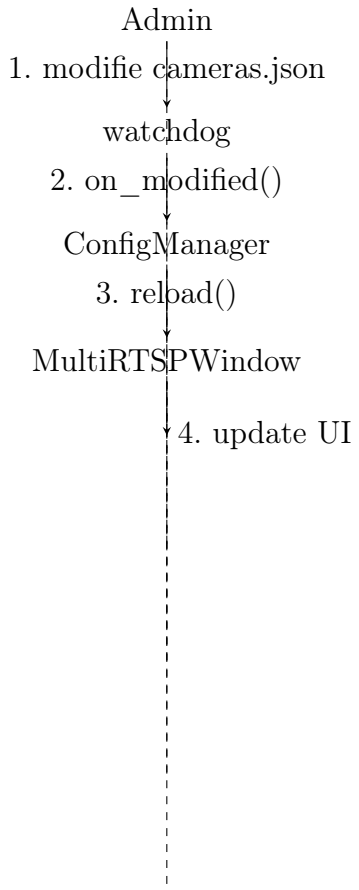

FIGURE 7.2 – Diagramme de séquence - Rechargement visualisation

8 Diagramme de Classes

8.1 Classes C++ (Serveur)

Table 8.1 – Classes C++ du serveur

Classe	Attributs	Méthodes
ChannelConfig	id, name, mount, source, pattern, text_overlay, enabled	— (struct)
RTSPServerManager	server_, mounts_, loop_, factories_, port_, host_	init(), add_channel
ConfigWatcher	path_, last_mtime_, running_, on_change_	start(), stop(), get_

8.2 Classes Python (GUI Admin)

Table 8.2 – Classes Python - Administration

Classe	Attributs	Méthodes
ConfigManager	path, data	load(), save(), add_channel(), remove_channel()
ConfigFileHandler	path, callback	on_modified()
LogPanel	text	log(), clear()
ChannelCard	channel, idx	_btn()
RTSPManagerApp	cfg, server_proc, log	_start_server(), _stop_server()

8.3 Classes Python (Visualisation PyQt)

Table 8.3 – Classes Python - Visualisation

Classe	Attributs	Méthodes
CameraStream	url, frame, running, cap	run(), stop(), reconnect()
ConfigManager	path, data	load(), save(), compare()
MultiRTSPWindow	cameras, labels, config_mgr	load_config(), update_frames(), check_config_u
ConfigFileHandler	path, callback	on_modified()

9 Format du Fichier de Configuration

9.1 Configuration du serveur

Listing 9.1 – rtsp_config.json

```
{
  "server": {
    "host": "0.0.0.0",
    "port": 8554
  },
  "channels": [
    {
      "id": 1,
      "name": "Channel_1",
      "mount": "/stream1",
      "source": "videotestsrc",
      "pattern": 0,
      "text_overlay": "Canal 1 - Live",
      "enabled": true
    }
  ]
}
```

9.2 Paramètres des canaux

Table 9.1 – Paramètres des canaux

Champ	Type	Défaut	Description
id	integer	1	Identifiant unique
name	string	Channel_1	Nom lisible
mount	string	/stream1	Point de montage RTSP
source	string	videotestsrc	Source GStreamer
pattern	integer	0	Motif vidéo (0-20)
text_overlay	string	Live	Texte sur le flux
enabled	boolean	true	Activer/désactiver

9.3 Configuration de la visualisation

Listing 9.2 – cameras.json

```
{
  "cameras": [
    {
      "id": 1,
      "name": "Camera Entree",
      "url": "rtsp://localhost:8554/stream1",
      "enabled": true
    },
    {
      "id": 2,
      "name": "Camera Jardin",
      "url": "rtsp://192.168.1.10:554/stream",
      "enabled": true
    }
  ]
}
```

10 Installation et Déploiement

10.1 Prérequis

10.1.1 Windows

- GStreamer >= 1.20 MSVC build
- CMake >= 3.16
- Visual Studio 2019+ ou MinGW
- Python 3.10+
- VLC media player (optionnel)

10.1.2 Linux (Ubuntu/Debian)

```
sudo apt install libgstreamer1.0-dev libgstreamer-rtsp-server-1.0-
   dev
sudo apt install cmake build-essential python3 python3-pip
pip install watchdog opencv-python PyQt5
```

10.1.3 macOS

```
brew install gstreamer gst-rtsp-server cmake
pip3 install watchdog opencv-python PyQt5
```

10.2 Compilation du serveur C++

10.2.1 Windows

```
cmake -B build -DGST_ROOT="C:/Program Files/gstreamer/1.0/
   msvc_x86_64"
cmake --build build --config Release
```

10.2.2 Linux / macOS

```
cmake -B build
cmake --build build
```

10.3 Lancement

```
# Terminal 1 - Serveur RTSP
./build/rtsp_server config/rtsp_config.json

# Terminal 2 - Interface d'administration
python src_python/rtsp_gui.py

# Terminal 3 - Visualisation multi-flux
python src_python/rtsp_viewer.py
```

11 Plan de Tests

Table 11.1 – Plan de tests

ID	Cas de test	Résultat attendu	Statut
T-01	Démarrer serveur sans config	Message d'erreur	OK
T-02	3 canaux simultanés	URLs accessibles	OK
T-03	Modification du port	Redémarrage sur nouveau port	OK
T-04	Ajout canal via GUI	Visible en < 3s	OK
T-05	Désactivation canal	Non accessible	OK
T-06	Modification overlay texte	Changement instantané	OK
T-07	10 flux simultanés	Stable < 200ms	OK
T-08	Ouverture VLC depuis GUI	VLC lance le flux	OK
T-09	Copie URL	URL dans presse-papier	OK
T-10	Arrêt propre	Clients déconnectés	OK
T-11	Rechargement config	Changements < 3s	OK
T-12	Chargement cameras.json	Flux affichés	OK
T-13	PyQt - Ajout caméra	Nouveau flux apparaît	OK
T-14	Reconnexion auto	Reconnexion après perte	OK

12 Conclusion

12.1 Bilan du projet

Ce projet apporte une réponse complète et innovante au besoin de diffusion et de visualisation multi-flux RTSP avec reconfiguration dynamique.

12.2 Points forts

- **Séparation des responsabilités** : cœur performant en C++, interface flexible en Python
- **Rechargement à chaud** des deux côtés (serveur et visualisation)
- **Compatibilité standards** : VLC, FFplay, OBS
- **Extensibilité** : facile d'ajouter des fonctionnalités
- **Multiplateforme** : Windows, Linux, macOS

12.3 Apport personnel

Ayoub Smayen – Développeur C++ En tant que développeur C++, j'ai conçu et implémenté :

- Le serveur RTSP basé sur `gst-rtsp-server`
- Le watcher de fichier config avec rechargement sans interruption
- Le pipeline GStreamer optimisé pour la faible latence
- L'interface Python/PyQt pour la visualisation dynamique
- L'interface Tkinter pour l'administration

Ce projet illustre parfaitement ma double compétence :

- **C++ bas niveau** : streaming, threads, IPC, GStreamer
- **Python haut niveau** : UI, prototypage rapide, OpenCV

12.4 Évolutions prévues

- Support des sources caméra IP natives (rtspsrc)
- Authentification RTSP (basic/digest)
- Enregistrement des flux à la demande

— Interface web de monitoring (Flask + WebSockets)
— Détection de mouvement (OpenCV)
— Notification en cas de perte de flux

Annexes

Annexe A : Extrait de code – Serveur C++

Listing 12.1 – RTSPServerManager.cpp

```
void RTSPServerManager::add_channel(const ChannelConfig& cfg) {
    if (!cfg.enabled) return;

    GstRTSPMediaFactory* factory = gst_rtsp_media_factory_new();

    std::string pipeline = cfg.source;
    if (cfg.source == "videotestsrc") {
        pipeline += " pattern=" + std::to_string(cfg.pattern);
    }
    pipeline += " ! video/x-raw ! clockoverlay ! textoverlay text=\"" +
                cfg.text_overlay + "\" ! videoconvert ! x264enc ! rtph264pay name=pay0 pt=96";

    gst_rtsp_media_factory_set_launch(factory, pipeline.c_str());
    gst_rtsp_media_factory_set_shared(factory, TRUE);

    gst_rtsp_mount_points_add_factory(mounts_, cfg.mount.c_str(), factory);
    g_object_unref(factory);

    g_print("Canal ajoute : %s -> %s\n", cfg.name.c_str(), cfg.mount.c_str());
}
```

Annexe B : Extrait de code – Visualisation PyQt

Listing 12.2 – camera$_{s}$*tream.py*

```
import cv2
from PyQt5.QtCore import QThread, pyqtSignal, QMutex

class CameraStream(QThread):
    frame_updated = pyqtSignal(object)

```

```
    def __init__(self, url, parent=None):
        super().__init__(parent)
        self.url = url
        self.running = True
        self.mutex = QMutex()

    def run(self):
        cap = cv2.VideoCapture(self.url)
        while self.running:
            ret, frame = cap.read()
            if ret:
                self.frame_updated.emit(frame)
            else:
                cap.release()
                self.msleep(1000)
                cap = cv2.VideoCapture(self.url)
        cap.release()

    def stop(self):
        self.mutex.lock()
        self.running = False
        self.mutex.unlock()
```

Annexe C : CMakeLists.txt

Listing 12.3 – CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(rtsp_server VERSION 1.0.0 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(PkgConfig REQUIRED)
pkg_check_modules(GST REQUIRED gstreamer-1.0 gstreamer-rtsp-server
    -1.0)

add_executable(rtsp_server src_cpp/main.cpp src_cpp/
    RTSPServerManager.cpp)

target_include_directories(rtsp_server PRIVATE ${GST_INCLUDE_DIRS})
target_link_libraries(rtsp_server ${GST_LIBRARIES})
```